

BDI Commit Message Conventions

Overview

This project follows the [Conventional Commits](https://www.conventionalcommits.org/) (<https://www.conventionalcommits.org/>) specification for commit messages. This provides a consistent format that enables automated changelog generation and semantic versioning.

Commit Message Format

```
<type>(<scope>): <subject>

<body>

<footer>
```

Components

Type (Required)

The type describes the kind of change being made:

- **feat**: A new feature
- **fix**: A bug fix
- **docs**: Documentation only changes
- **style**: Changes that do not affect the meaning of the code (white-space, formatting, missing semi-colons, etc)
- **refactor**: A code change that neither fixes a bug nor adds a feature
- **perf**: A code change that improves performance
- **test**: Adding missing tests or correcting existing tests
- **build**: Changes that affect the build system or external dependencies
- **ci**: Changes to CI configuration files and scripts
- **chore**: Other changes that don't modify src or test files
- **revert**: Reverts a previous commit

Scope (Optional)

The scope provides additional contextual information about what part of the codebase is affected:

Module Scopes:

- `bci` : Binary Computational Interface
- `btl` : Binary Translation Layer
- `compiler` : Compiler toolchain (lexer, parser, analyzer, codegen)
- `vm` : Virtual machine
- `kernel` : Kernel layer
- `device` : Device management
- `scheduler` : Task scheduler
- `fs` : File system

- backend : Hardware backends (GPU, FPGA, BPU)
- ai-trainer : AI training infrastructure

Component Scopes:

- lexer : Lexical analyzer
- parser : Parser
- analyzer : Semantic analyzer
- codegen : Code generator
- tests : Test suite
- docs : Documentation
- build : Build system

Subject (Required)

The subject contains a succinct description of the change:

- Use the imperative, present tense: “change” not “changed” nor “changes”
- Don’t capitalize the first letter
- No period (.) at the end
- Maximum 50 characters

Body (Optional)

The body provides additional context about the change:

- Use the imperative, present tense
- Include motivation for the change
- Contrast with previous behavior
- Wrap at 72 characters

Footer (Optional)

The footer contains information about breaking changes and issue references:

- **Breaking Changes:** Start with BREAKING CHANGE: followed by description
- **Issue References:** Fixes #123 , Closes #456 , Refs #789
- **Reviewed-by:** Reviewed-by: Name <email>
- **Signed-off-by:** Signed-off-by: Name <email>

Examples

Feature Addition

```
feat(compiler): add nullptr support to lexer
```

Implement C23 nullptr keyword recognition **in** the lexical analyzer. The lexer now properly tokenizes nullptr **and** distinguishes it from NULL macro usage.

- Add NULLPTR token **type**
- Update keyword table
- Add test cases **for** nullptr tokenization

Refs **#**42

Bug Fix

`fix(vm)`: resolve stack overflow in recursive calls

The VM stack was not properly checking depth limits, causing crashes on deeply recursive function calls. This fix adds proper stack depth validation and returns an error when the limit is exceeded.

The stack depth limit is now configurable via `VM_MAX_STACK_DEPTH`.

Fixes #123

Documentation Update

`docs(api)`: update device API contracts

Add detailed preconditions and postconditions for `device_init()` and `device_execute()` functions. Include thread safety guarantees and error code documentation.

Refactoring

`refactor(kernel)`: modernize scheduler **with** C23 features

Replace `NULL` **with** `nullptr` throughout scheduler implementation.

Add `[[nodiscard]]` attributes **to** functions returning error codes.

Use `typeof` **for** type inference **in** generic macros.

No functional changes, purely modernization **to** C23 standard.

Performance Improvement

`perf(backend)`: optimize GPU kernel launch overhead

Reduce kernel launch latency by 40% through:

- Batch kernel parameter uploads
- Use pinned memory **for** transfers
- Implement kernel launch queue

Benchmark results show improvement from 150µs to 90µs average launch time.

Breaking Change

feat(bci)!: change API to use opaque handles

BREAKING CHANGE: The BCI API now uses opaque handles instead of exposing internal structures. This improves encapsulation and allows for future ABI stability.

Migration guide:

- Replace direct struct access with getter functions
- Use `bci_handle_create()` instead of manual allocation
- Call `bci_handle_destroy()` to free resources

Old code:

```
BciContext* ctx = malloc(sizeof(BciContext));
ctx->device_id = 0;
```

New code:

```
BciHandle* handle = bci_handle_create();
bci_set_device(handle, 0);
```

Fixes #200

Multiple Changes

feat(compiler): implement C23 attribute annotations

Add support for C23 standard attributes:

- `[[nodiscard]]` for functions returning important values
- `[[maybe_unused]]` for intentionally unused parameters
- `[[noreturn]]` for functions that never return
- `[[fallthrough]]` for intentional switch fallthrough

The parser now recognizes attribute syntax and the semantic analyzer validates attribute usage. Code generator emits appropriate compiler-specific attributes.

- Add attribute parsing to parser
- Implement attribute validation in analyzer
- Update code generator for attribute emission
- Add 50+ test cases for attribute handling

Refs #150, #151, #152

Revert

revert: feat(vm): add JIT compilation support

This reverts commit a1b2c3d4e5f6g7h8i9j0.

The JIT implementation caused stability issues on ARM platforms. Reverting until the issues can be properly diagnosed and fixed.

Refs #300

Commit Message Best Practices

DO

- ✓ **Use imperative mood:** “add feature” not “added feature”
- ✓ **Be concise:** Keep subject line under 50 characters
- ✓ **Be specific:** Describe what changed and why
- ✓ **Reference issues:** Link to relevant issue numbers
- ✓ **Explain context:** Use body for detailed explanation
- ✓ **One logical change:** Each commit should be atomic
- ✓ **Test before commit:** Ensure code compiles and tests pass

DON'T

- ✗ **Don't be vague:** “fix bug” or “update code”
- ✗ **Don't combine unrelated changes:** Keep commits focused
- ✗ **Don't commit broken code:** Each commit should be buildable
- ✗ **Don't use past tense:** “added” or “fixed”
- ✗ **Don't exceed line limits:** 50 for subject, 72 for body
- ✗ **Don't forget scope:** Helps with changelog generation
- ✗ **Don't skip the body:** Explain non-obvious changes

Commit Workflow

1. Stage Changes

```
# Stage specific files
git add C/compiler/lexer/bci_lexer.c
git add C/compiler/lexer/bci_lexer.h

# Stage all changes (use carefully)
git add -A
```

2. Write Commit Message

```
# Open editor for commit message
git commit

# Or provide message inline (for simple commits)
git commit -m "feat(lexer): add nullptr token support"
```

3. Review Commit

```
# View commit details
git show

# View commit log
git log --oneline -5
```

4. Amend if Needed

```
# Amend last commit (before pushing)
git commit --amend

# Change commit message only
git commit --amend -m "new message"
```

Commit Hooks

Pre-commit Hook

Create `.git/hooks/pre-commit`:

```
#!/bin/bash

# Run tests
make test
if [ $? -ne 0 ]; then
    echo "Tests failed. Commit aborted."
    exit 1
fi

# Check formatting
make format-check
if [ $? -ne 0 ]; then
    echo "Code formatting issues. Run 'make format' and try again."
    exit 1
fi

exit 0
```

Commit-msg Hook

Create `.git/hooks/commit-msg`:

```
```bash
```

## #!/bin/bash

```
commit_msg_file=$1
commit_msg=$(cat "$commit_msg_file")
```

## Check commit message format

```
if ! echo "$commit_msg" | grep -qE "^(feat|fix|docs|style|refactor|perf|test|build|ci|chore|revert)((.
+))?: .+"; then
 echo "Error: Commit message does not follow conventional commits format"
 echo ""
 echo "Format: ():
```